Spending Fewer Tokens

Every way to make Claude Code and Golden Thread use less of your allowance: what to turn on, how, what you will notice when it is on, and what you give up.

A Golden Thread guide, written against gt 0.22.1 (figures not re-measured for 0.22.0 are marked 0.21.4). Figures are labelled: measured (a run or a recorded event produced it, on the author's machine), estimated (worked out from a measurement), or not measured. None has been reproduced by anyone else yet.

One setting for all of it: `usage_plan` *(new in 0.22.1)*

Gt has a dozen settings that save tokens, and they work best when they agree with each other. `usage_plan` is one switch for the group, so you do not have to set them one by one:

USAGE_PLAN	WHAT IT DOES
`off`	No plan. Every setting is yours. **Choosing `off` puts every setting back to what it was before the plan changed it** (a setting you had never set is removed, so its default applies again).
`min`	The safe few. Nothing is cut while you are working.
`max`	`min` plus everything else in the table below. `auto_clear` is only ever set to `shadow` (it logs and never clears), because it has to prove itself first.
`custom`	Your own mix. Shown automatically when you change a setting the plan set: `custom (based on max)`. Change it back and it reads `max` again.

`min` and `max` save your current values the first time you leave `off`, then write the group. This is exactly what each one sets (generated from the same table the command uses, so it cannot drift):

SETTING	DEFAULT	MIN	MAX	WHAT IT DOES
`token_profile`	`standard`	unchanged	`minimum`	Cheapest answers: terse replies, no skeptic pass, no vault hints, the average model profile.
`farm_first`	`suggest`	unchanged	`yes`	Hand routine work to your own local engines by itself (needs the farm module).
`farm_ladder`	`off`	unchanged	`on`	Route work through the engines you calibrated, cheapest first (needs the farm module).
`farm_share`	`off`	unchanged	`on`	Send a share of eligible tasks to hosted or local engines (needs the farm module).
`cut_handoff`	`facts`	`facts`	`facts`	What a cut carries into the fresh session: the findings and key facts.
`cut_kind`	`compact`	`clear`	`clear`	A cut is a clear with that handoff, not a /compact.
`expiry_guard.handoff_pickup`	`on`	unchanged	`on`	After a clear, load the saved handoff into the fresh session by itself (nothing to press).
`auto_compact`	`warn`	unchanged	`250000`	Cut the session by itself once its context reaches this many tokens.
`auto_clear`	`off`	unchanged	`shadow`	Score after each reply whether clearing would pay; only logs, never clears.
`cut_advisor`	`off`	`suggest`	`suggest`	Say once when a cut would pay back; never cuts.
`expiry_guard`	`warn-only`	`auto`	`auto`	Cut an idle session while its prompt cache is still warm.
`context_nudge`	`40`	`30`	`30`	Say once when the context passes this percent of the window.

A setting whose module is off (`farm_ladder` and `farm_share` need the farm module) is skipped and listed, never an error. Everything on the table has a default you can read with `/gt:gt-settings <name>` .

Turn it on and off: `/gt-usage:usage-plan max` (or `min` , `off`), or `gt_settings.py set usage_plan max` .

See exactly what it changed on this machine: `/gt-usage:usage-plan changes` (or `gt_settings.py usage-plan changes`) prints a table of setting, what it was, what it is now, its default, what it does, and the command that undoes it. It also marks any setting you have changed by hand since. `/gt-usage:usage-plan show` lists every member with its value now and its on and off commands. The same table is printed the moment you choose `min` or `max` .

Not the same as `token_profile` . `token_profile` trades answer quality against tokens, and its `maximum` spends *more*. `usage_plan max` is about saving, and it includes `token_profile minimum` . They share a word and nothing else.

WHAT EACH SETTING DOES, AND HOW TO TURN IT ON AND OFF

SETTING	WHAT IT DOES	ON	OFF
`auto_compact`	Cuts the session by itself when it grows large. `warn` (default) says so once and never cuts.	`set auto_compact auto` (150,000 tokens), `250000`, or a percent `30` - `90`	`set auto_c` `off`
`cut_kind`	What a cut is: `compact` (summarise in place) or `clear` (a fresh session that starts from the handoff).	`set cut_kind clear`	`set cut_ki`
`cut_handoff`	What a cut saves first: `facts` (findings, a handoff and key facts), `handoff`, or `none`. A clear needs one that is not `none`.	`set cut_handoff facts`	`set cut_ha`
`auto_clear`	After each reply, scores whether clearing now would pay back. `shadow` only logs; `on` clears for you once the log has proved the rule (it asks you to type a word in a terminal).	`set auto_clear shadow`	`set auto_c`
`cut_advisor`	Says once when a cut would pay back, and never cuts. It stays quiet while `auto_clear` is on or `auto_compact` is cutting for you (0.22.1).	`set cut_advisor suggest`	`set cut_ad`
`expiry_guard`	Cuts an idle session while its prompt cache is still warm, so coming back is not a full rebuild.	`set expiry_guard auto`	`set expiry`
`context_nudge`	One line when this session passes a share of its window.	`set context_nudge 30`	`set contex`
`token_profile`	The cheapest answer style and models: `minimum` is terse and cheapest, `standard` the default, `maximum` spends most.	`set token_profile minimum`	`set token_`
`reply_length`	How long Claude's replies are: `terse`, `brief`, `normal`. A part of `token_profile`.	`set reply_length terse`	`set reply_`
`expiry_guard.handoff_pickup`	After a clear, loads the saved handoff into the fresh session by itself, so a cut needs nothing from you. With it off, the pickup line is put in the prompt box and you press Enter.	`set expiry_guard.handoff_pickup on` (default)	`set expiry`
`expiry_guard.pickup_hold`	After a clear hands the fresh session its handoff, no automatic cut happens until you send that session's first prompt, which turns cuts back on. Stops a session nobody has touched from clearing and reloading in a loop.	`set expiry_guard.pickup_hold first_prompt` (default)	`set expiry`
`farm_first`	Hands routine work to your own local engines.	`set farm_first yes`	`set farm_f`
`farm_ladder`	Routes work through the engines you have calibrated, cheapest first (farm module).	`set farm_ladder on`	`set farm_l`
`farm_share`	Sends a share of eligible tasks to engines (farm module). Set it up with `/gt-usage:usage-plan farm --base-url URL`, which calibrates the engine and turns this on only if it passes.	`set farm_share on`	`set farm_s`

Each of these has a longer explanation with `gt_settings.py explain <name>`.

HOLD: NO AUTOMATIC CUT IN THIS SESSION

Automatic cuts are the right default for a long day and the wrong one when you are mid-debug with exact values in the context. **Hold** stops all four automatic cuts for one session: `auto_clear`, `auto_compact`, the idle-session guard (`expiry_guard`) and the open-session path.

- **During a session:** `/gt-usage:usage-plan hold` (and `hold off` to end it; `hold status` says whether it is on).
- **At the start of a session:** launch with `GT_HOLD_CUTS=1 claude`. The session is held from its first prompt (the first hook that runs records it). Only one session can be held at a time, and a launch-time hold holds nothing while another live session's hold exists. The cut advisor still speaks in a held session, because nothing will cut it.

- **It ends with `hold off`, `/clear` or a new session.** Resuming the same session keeps it. A held session that sits idle loses the cache-expiry protection, so coming back to it later can cost a full rebuild of its context.

Your sessions burn allowance fast? Do these, in this order

1. **Close or compact long sessions early.** This is the biggest lever by far. Leave `context_nudge` at 40 so gt tells you when a session has grown, and leave `auto_compact` on its default (`warn` : gt says once when a session passes 150,000 tokens and never compacts by itself), or opt in with `auto_compact auto` (gt saves your work and compacts by itself at 150,000 tokens; `auto_compact 50` cuts earlier). Two more settings shape each cut: `cut_handoff` (`facts` , the default, saves findings and a handoff with its key facts; `handoff` a handoff only; `none` a plain cut) and `cut_kind` (`compact` , the default, or `clear` : a fresh session that starts from the handoff). When you finish a piece of work, say "I'm done for now" (`/gt:gt-close`) and start a fresh session.
2. **Let gt trim sessions you walk away from.** Set `expiry_guard auto` (the `standard` and `minimum` token profiles do this for you).
3. **Shrink what every session starts with.** Keep `startup_output compact` (the 0.21.4 default), disable the gt plugins you do not use, and keep your CLAUDE.md files and memory index short.
4. **Keep big text out of the conversation.** Turn on `spill_threshold_lines 200` and `paste_intake ask` , and use `/gt:gt-digest` for a big file or log.
5. **Pick a token profile.** `standard` is the default set; `minimum` spends least.
6. **If you run a model on your own computer,** let it answer questions about your notes (`vault_answers local-first`) and take routine work (`farm_first`).
7. **For long working sessions,** consider `rules_injection compact` .
8. **After a week, check what each mechanism actually saved** with `/gt-usage:gt-usage-savings` . Keep what pays, turn off what does not.

To change any setting, type `/gt:gt-settings <name> <value>` in Claude, or just say it ("set auto_compact to 50"). `/gt:gt-settings` on its own shows them all with their current values.

Where your tokens actually go

Every time Claude replies, Claude Code sends it the **whole conversation again**: every message, every file Claude read, every command output. A prompt cache makes the repeats cheaper, but they are still paid for, on every turn, until the conversation is cut.

On a real multi-agent build, gt measured where the cost went (**measured**, 2026-10-15):

PART	SHARE OF THE COST
Re-reading the conversation from the cache	73–74%
Writing new text into the cache	24–26%
Claude's written output	0.3–1.5%
Claude's thinking (part of the output)	0.1–0.9%

So **cost is roughly the number of turns times the size of the conversation.** A long session costs more per message the longer it runs. Everything in this guide does one of four things: fewer turns per session, a smaller conversation on each turn, less big text pasted or printed into it, or handing the work to something cheaper.

What barely matters: how long Claude's replies are, and its effort level. Cutting out all of Claude's thinking would save about 1% (**estimated**). Those settings change speed and how much you read, not your allowance.

1. Keep sessions short

In the Setting column, type `/gt:gt-settings` followed by what is shown, for example `/gt:gt-settings auto_compact 50` , or just say it.

LEVER	WHAT YOU WILL NOTICE	WHAT IT SAVES	SETTING
Context nudge	At 40% of the context window, one line says the session is large and suggests closing it and starting fresh; again at 55, 70 and 85%. Claude mentions it once, at a break. Nothing is blocked.	Nothing by itself; the saving is what you do next. **Not measured.**	`context_nudge 40` (default), `30` earlier, `off`
Auto-compact	Default `warn`: when a turn ends with the session past 150,000 tokens, one line says it is large and names `/gt:gt-minimize compact`; nothing is compacted and your prompt box is left alone. Opted in: when a turn ends with the session past the size you chose, Claude first saves new findings to your notes and writes a handoff, then the session is compacted in place and "Read the handoff at ..." appears in your prompt box. Press Enter to carry on.	Everything removed is not re-read on any later turn. **Not measured** on a live session.	`warn` (default: a notice at 150,000 tokens, held to 75% of the model's window when known); opt in with `auto` (a cut at that size), `auto_compact 50` (a percent, 30–90) or a token count such as `150k`; `off` (neither)
Close and start fresh	Say "I'm done for now" (`/gt:gt-close`): gt saves what the session learned and releases the project. The next session starts small and reads the notes, not the old conversation.	A fresh session starts near 40K tokens (**measured** on the author's machine) instead of carrying hundreds of thousands.	Any time
Compact with a handoff	`/gt:gt-minimize compact` does the auto-compact steps once, now. Plain `/compact` also works but does not save to your notes first.	As auto-compact.	Any time
What a cut carries, and what kind	Every cut gt makes (opted-in auto-compact, `/gt:gt-minimize compact`) follows two settings. `cut_handoff`: `facts` saves new findings to your notes and writes a handoff with its key facts (as before); `handoff` writes the handoff only; `none` is a plain cut with nothing carried over. `cut_kind`: `compact` shrinks the session in place; `clear` ends it and puts "Read the handoff at ..." in the fresh session's prompt box (if gt cannot clear by itself it asks you to type `/clear` and fills the box after). `clear` with `none` is refused.	In the owner's cut-savings tests a clear with a handoff and its facts saved the most in the most sets (**self-verified** on the owner's test sets; **not measured** live through gt). The clear itself has run only against a fake Claude Code in gt's tests.	`cut_handoff facts` (default), `handoff`, `none`; `cut_kind compact` (default), `clear`
Idle-session guard	`warn-only`: a desktop notice when a big idle session is about to go cold. `auto`: gt also saves, hands off and compacts the idle session before its cache lapses, so you come back to a small session.	On the author's six days: about 8.5M token-equivalents kept, 70% of what cold resumes would have cost (**estimated**). Helps people who step away; does little if you are at the keyboard all day.	`expiry_guard auto`, `warn-only` (default), `off`

What you give up. A compacted session is a summary of the old one, not every word. gt saves findings and a handoff first so nothing you would want is lost, but the exact wording of an early message is gone from the conversation.

The context nudge reads the session size from gt's status line (the gt-usage module); a default install sets no status line, so without it the nudge never speaks (add it with `python3 ~/.claude/golden-thread/hooks/gt_usage.py --setup-statusline`, or re-run the installer with `--statusline yes`). Auto-compact does not need the status line: it reads the session size from Claude Code itself, and runs only in terminal Claude Code.

2. Shrink what every turn carries

Some text rides along on every single turn: gt's start-up report, its rules, your CLAUDE.md files, the memory index, and the list of every installed skill and agent. Each token of it is re-read on every turn.

The author's own trim (measured, single runs, 2026-10-15): turning on the token-diet settings below and shortening the memory index and the vault's CLAUDE.md took the first turn of a session from **41,756 to 39,031 tokens**, and cut what each later turn adds from **about 1,351 to about 813 tokens**.

In the Setting column, type `/gt:gt-settings` followed by what is shown, for example `/gt:gt-settings auto_compact 50`, or just say it.

LEVER	WHAT YOU WILL NOTICE	WHAT IT SAVES	SETTING
Short start-up report (new in 0.21.4)	When everything is healthy, the session-start checks share one line, e.g. "gt 0.21.4 current · components clean · workers clean". A check spells itself out only when something needs you. `/gt:gt-doctor` still prints the full report.	3,701 → 1,486 characters on a test fixture (**measured**), about 550 tokens per session (**estimated**).	`startup_output compact` (default), `full`
Rules sent once	**No visible change.** gt sends its rules to Claude in full at the start, after a compaction and every 20 messages; in between, one line says they are unchanged. In 0.21.4 that line also restates the timestamp rule. The checks that catch a broken rule still run every turn.	A full-mode injection is about 436 tokens per prompt (1,743 bytes / 4, **estimated**); compact saves about 340 tokens per message after the first on a fresh vault (**estimated**; measured by an independent agent on a clean macOS 26.6.2 VM, 25 Oct 2026); in compact mode gt falls back to the full text when it finds no transcript. In a 90-run test on 2026-10-15, 0.969× the cost of full per correct result (**measured**).	`rules_injection compact`; `full` (default)
Short CLAUDE.md files	New projects get a short CLAUDE.md stub (2,296 → 377 bytes, **measured**). When the project settles, `/gt:gt-brief` drafts the full one for you to review.	The difference, on every turn, for as long as the project is open.	Automatic for new projects in 0.21.4
Trim your own CLAUDE.md and memory	Nothing changes in how Claude works, as long as instructions stay and history goes. `/gt:gt-optimize` lists duplicated facts, dead index rows and memory over budget, and can move them out with a pointer left behind.	The author cut the memory index from 11,600 to 6,302 characters and the vault CLAUDE.md from 6,702 to 2,668 (**measured**).	Run `/gt:gt-optimize`, then edit
Disable plugins you do not use	That plugin's commands, agents and tools disappear from Claude until you enable it again (from the next session).	See the table below.	A terminal command; see below
Smaller skill list (0.21.4)	The old alias commands are gone: use `/gt:gt-create task`, `/gt:gt-handle task`, `/gt:gt-list tasks`, and the same with `handoff`. Skill and agent descriptions are shorter.	Smaller on every turn; figures in the CHANGELOG.	Automatic

Why the timestamp line matters. In the 90-run test of rules sent once (2026-10-15), Claude started its replies with a timestamp 84% of the time against 93% with the rules sent in full (**measured**). The rest held: tests before commits, no leaked keys. That is why 0.21.4 repeats the timestamp rule in the one-liner. Whether that closes the gap is **not measured** yet.

Keep the rules on. A separate 90-run test (2026-10-11) compared rules on with rules off: with them off, a correct result cost **1.26×** as much, because Claude committed with failing tests and had to redo work; and a live key was pasted into the reply in 7 of 15 runs, against 0 of 15 with the rules on (**measured**). In one sentence: a correct result cost 1.26× as much with the rules off (90-run A/B benchmark on Opus, Sonnet and Haiku, 2026-10-11; every failed run re-prompted until correct; weighted by a real model mix). The rules pay for themselves; sending them once is the cheaper way to keep them.

WHAT EACH GT PLUGIN ADDS TO EVERY SESSION

Claude Code can tell you itself: `claude plugin details <name>@golden-thread-plugin` prints a plugin's "always-on" cost. On 25 Oct 2026 an independent agent on a clean macOS 26.6.2 VM read gt 0.22.0's core at about 3,080 tokens and all 12 plugins together at about 5,470 (Claude Code's own report; MCP tool schemas are not counted). The other rows below are the 0.21.4 figures, not re-measured for 0.22.0 (**estimated** by Claude Code):

PLUGIN	ALWAYS-ON	DISABLE IT IF
gt (core)	~3,080 (0.22.0, measured by an independent agent, 25 Oct 2026)	never: this is Golden Thread
gt-lotr	~2,000, plus its tools	you do not reach Jira, mail, GitHub or other systems through gt
gt-visualize	~1,100	you do not make 3D explainers or code cities
gt-wiki	~700	you do not keep a wiki
gt-pane	~290	you do not use the button pane
gt-workflow	~290	you do not run saved workflows
gt-usage	~280	(keep it: the context nudge needs its status line)
gt-farm	~240	you have no local engines
gt-demo	~200	you do not give demos
gt-flow, gt-watch	~190 each	you do not use them

To turn one off, and back on, in a terminal (it takes effect from the next session; inside Claude Code, `/plugin` does the same):

```
claude plugin disable gt-lotr@golden-thread-plugin
claude plugin enable gt-lotr@golden-thread-plugin
```

`claude plugin details` does not count a plugin's MCP tools. On the 0.22.0 check above, gt-lotr's tool list came to **about 2,130 tokens** (**estimated**; measured by an independent agent on a clean macOS 26.6.2 VM, 25 Oct 2026). On the author's machine in 0.21.4, gt-visualize plus gt-demo came to **about 900** (**estimated**, not re-measured).

SETTINGS THAT ADD A LINE TO EVERY TURN

Some settings are free. Others add one line of text to every turn, and some spend whole agents.

COSTS NOTHING (SHOWN ONLY TO YOU)	ADDS A LINE PER TURN	SPENDS AGENTS
`prompt_hints`: a "Tip:" line naming the command for what you typed	`vault_hints`: up to three matching note titles	`skeptic_pass`: an agent checks findings at each write-back
`next_step_hints`: a "Next:" line after a reply	`hints_as_choices`: next steps offered as a question after a gt command	`model_calibration report`: re-runs agent work on other models, within a budget
	`reply_length brief` or `terse`: one line, but shorter replies	
	`allowance_note`: one line, only once you pass the threshold	

Turn any of them off with `/gt:gt-settings <name> off`. The per-turn lines are small (one line each); the agents are the bigger cost and also the ones that catch mistakes.

3. Keep big text out of the conversation

A 12,000-token paste costs about 15,000 token-equivalents when it arrives and about 1,200 more on every turn after it (**estimated**, from the cache prices). Big text is cheapest when it never enters.

In the Setting column, type `/gt:gt-settings` followed by what is shown, for example `/gt:gt-settings auto_compact 50`, or just say it.

LEVER	WHAT YOU WILL NOTICE	WHAT IT SAVES	SETTING
Long output kept out	When a command prints more than the line count you set, Claude sees only the first and last lines, any lines that look like errors, and where the full output was saved. It reads the file if it needs more. A failing command is not touched.	The rest of the output, on every later turn. **Not measured** live.	`spill_threshold_lines 200`; `off` (default)
Big pastes held back	A message longer than the limit is saved to a private file and not sent. One line gives the saved path and suggests `/gt:gt-digest <file> <what you want>`. Send the same message again within 3 minutes and it goes through unchanged; with no prompt history (Claudian, another SDK app) paste it again or type `read <file>`. A headless run (`claude -p`) is never held. A pasted image is not caught. The limit is `paste_intake_chars`, 20,000 characters by default, when a local engine is set up; with no local engine it is raised to the break-even size, 72,000 characters (18,000 tokens at 4 characters a token) unless `break_even` is off or you set `break_even_tokens`.	The whole paste. **Not measured** end to end.	`paste_intake ask` (or `auto`); `off` (default)
Digest a file	`/gt:gt-digest <file> <question>`: a cheaper reader (your local engine, else a small helper agent, else Claude itself when the file is small) reads it and you get only the answer.	The file stays out of the conversation. **Not measured.**	Any time
Size rule for helpers	gt suggests a helper agent only when the reading is big enough to pay for it. You see shorter advice for small files.	A helper opens with about 18,000 tokens of its own before reading anything (**measured**, 27 runs), so a small read is cheaper inline.	`break_even on` (default), `off`

What you give up. With output kept out, Claude may need one extra step to read the saved file. Add `# gt:no-spill` to a command to keep its output whole.

4. Hand work to something cheaper

In the Setting column, type `/gt:gt-settings` followed by what is shown, for example `/gt:gt-settings auto_compact 50`, or just say it.

LEVER	WHAT YOU WILL NOTICE	WHAT IT SAVES	SETTING
Notes answered locally	When you ask what your notes say, the answer comes from the model on your own computer: a few sentences with the exact quoted lines and where they are. If the engine is unsure or a quote does not check out, Claude answers as it always did. If the engine is set up but stopped, gt asks once whether to start it; it never starts one on its own.	The vault pages are not read into the conversation; the short answer and its quoted lines are. Three real questions kept about 3,500 tokens out (**estimated**), taking 25–70 seconds each (**measured**).	`vault_answers local-first`; `off` (default). To stop the start question: `vault_answers_start never`
Routine work on your own engines	`suggest` (default): gt tells you what your engines could take and prints the command. `yes`: gt tests a new engine once, within your budget, sends it the work it passed, and tells the assistant every turn to hand any eligible inline task to the cheapest tested engine (a hosted one only for a packet marked public, after the privacy checks).	Local tokens cost no allowance. On the author's week, 84 packets (186,845 tokens) ran locally: about 0.1% of the work (**measured**).	`farm_first suggest`, `yes`, `off`
Import stages on a local engine	`/gt:gt-ingest` runs its extract, classify and draft steps on your big local engine; slower, and it falls back to Claude when the answer fails its checks.	Those steps' tokens. Quality was acceptable on 10 of 12 units, below Claude's (**measured**).	`pipeline_engine on`; `off` (default)
Model per skill	`average`: quick gt skills run on the small model, balanced ones on the middle model, deep ones on the large model. You may notice a slightly plainer answer from a quick skill.	On the author's six days, about 9.9% of agent work (**estimated**).	`models profile average`; `very-high`, `inherit` (default)
Token profile	One choice that sets seven settings at once (model profile, reply length, skeptic, note hints, idle guard, import helpers, calibration). `/gt:gt-settings` then shows the profile at the top, or "manual (based on ...)" once you change one of the seven yourself.	The sum of the seven.	`token_profile standard` (default), `minimum`, `maximum`

`minimum` turns off the skeptic and calibration and makes replies terse: the lowest cost, with the least checking. `maximum` runs every skill on the largest model.

Local engines are a modest lever today: most of the cost is long interactive turns that do not move. The guide *Farming to Your Own Hardware* covers setting one up.

A send must earn its keep (gt-farm, 0.21.4). Sending work to an engine also costs Claude tokens: the call, the packet when Claude wrote it, and reading the result. Each ledger line now records that as `claude_overhead`, and `gt_farm_send.py ledger` shows the **net** after it. From gt 0.22.0 the savings report's "Farmed packets" row is net the same way: it also takes off the output of a local review that reached your session and what a calibration's reference answers cost Claude, and a "Farmed savings by purpose" table under the main one shows what the net is made of. The overhead is an estimate (characters / 4, plus a fixed 220 tokens per send). Because of that, an automatic send of a packet under `farm_min_chars` (default 2000 characters) is skipped as "not worth sending" and stays in Claude; `0` turns the rule off, and a send you type is never skipped. The default is a first baseline from 88 sends on one machine (break-even near 1,360 characters), mostly probe traffic, so it is an estimate. An identical packet to the same engine and model is not sent twice inside `farm_reuse_days` (default 7; `--fresh` forces a send). Every packet is scanned for credentials before it leaves. A long send prints a "waiting" line every `farm_heartbeat_s` seconds (default 30; `0` is off).

HOSTED SPEND: SAMPLE AND HOW TO SET UP YOUR OWN

gt 0.22.0 ships a **sample**, not your setup: `templates/hosted-spend.sample.md` in the gt-farm module shows the two settings (`usage_external`, `farm_budgets`) and one service-table row for a made-up engine on a made-up host (`example-provider`, address ending `.invalid`). It holds no key, no account id and no real address, and gt does not read it.

What you get without doing anything else is an **estimate**: the tokens sent and received, at the rate in the engine's row. The status line says `(estimated)` and stays quiet until an engine is at 80% of its budget or was answered with a 429. gt 0.22.0 **does not read a provider's balance or billing API**; the `balance_source` column in the sample is reserved and ignored.

To set up your own: copy the row into your service table and change every value (name, address, model, rate from the provider's pricing page); put the engine's API key in a mode-600 file by a script you run (never in a session), named in `key_file`; then `gt_settings.py set farm_budgets "yourname=10"`. For a measured balance you would also need a **read-only billing key from the provider, kept in the secrets store** and never entering the session or the repository. gt does not use one yet.

5. Use helper agents well

A helper agent (a subagent) does its reading in its own conversation and hands back a short answer. That keeps your main conversation small, but the helper itself is not free.

- **Give a narrow brief and ask for a short return.** A helper that explores everything, or returns everything it read, moves the cost instead of saving it.
- **Parallel helpers cost the same as one after another** (**measured**, 2026-10-15), and finish 3–4 times sooner (**estimated**). Run independent work in parallel.
- **Never fold many jobs into one long conversation.** Doing all of a multi-agent build in one growing context was worked out at **2.35–5.4×** the cost (**estimated**).
- **Do not hand a small read to a helper.** Below about 18,000 tokens, reading it yourself is cheaper (see the size rule above).

Recommended profiles

SETTING	EVERYDAY	LONG BUILD SESSIONS	DEMO MACHINE
`token_profile`	`standard`	`standard`	`standard`
`context_nudge`	`40`	`30`	`40`
`auto_compact`	`auto`	`auto`	`off`
`expiry_guard`	`auto`	`auto`	`warn-only`
`startup_output`	`compact`	`compact`	`compact`
`rules_injection`	`full`	`compact`	`full`
`spill_threshold_lines`	`200`	`200`	`off`
`paste_intake`	`ask`	`ask`	`off`
`vault_answers`	`local-first` if you have an engine	`local-first` if you have an engine	`off`
Plugins	disable what you do not use	disable what you do not use	keep gt-demo; disable the rest you will not show

- **Everyday:** short sessions, close at a natural break. These settings cost nothing you would notice and keep long days cheap.
- **Long build sessions:** many turns, so per-turn savings add up: rules sent once and an earlier nudge pay most here.
- **Demo machine:** nothing should change under the audience's eyes. No auto-compact mid-demo and no held-back pastes; everything else stays at the defaults.

Measure your own

Your numbers will differ from the author's. To see them:

- **`/context`** inside a session lists what the conversation holds: system prompt, skills, agents, memory files, MCP tools and messages. `claude -p "/context"` prints the same for a new session without opening one.
- **`claude plugin details <name>@golden-thread-plugin`** shows one plugin's always-on cost.
- **`/gt-usage:gt-usage`** shows where you stand against the 5-hour and weekly allowance.
- **`/gt-usage:gt-usage-savings`** shows what each gt mechanism saved and cost, with a net figure; a mechanism losing tokens over 7 days is flagged.

The three-turn check (before and after a change). Run three short turns in one headless session and read each turn's size. Turn 1 is what a session starts with; the difference between turns 2 and 3 is what each turn adds.

```
claude -p "Say OK." --output-format json > t1.json
SID=$(python3 -c 'import json; print(json.load(open("t1.json"))["session_id"])')
claude -p "Say OK." --resume "$SID" --output-format json > t2.json
claude -p "Say OK." --resume "$SID" --output-format json > t3.json
for f in t1 t2 t3; do python3 -c 'import json,sys; u=json.load(open(sys.argv[1]))["usage"]
print(sys.argv[1], u["input_tokens"] + u["cache_creation_input_tokens"]
      + u["cache_read_input_tokens"])' $f.json; done
```

Run it in the folder you normally work in, change one setting, run it again, and compare. One run each is a rough reading; repeat it if the difference is small.

What we do not claim

- Every figure here comes from one person's machine and a handful of author-run tests. No other user has been measured.
- Several levers have no number yet: auto-compact on a live session, long output kept out, big pastes held back, and rules sent once over a long session.
- Local engines save little so far, and their answers were below Claude's quality in blind checks; gt audits them for that reason.

Appendix: how it works underneath

For readers who want the mechanics; nothing here is needed to use the settings.

- The rules, hints, allowance and reply-length lines are added by gt's per-message hook. With `rules_injection compact` it checks that the full text is really in the conversation before it sends the one-line version, and sends the full text again when a rule changes.
- The context nudge reads the session's size from the gt-usage status line (none is set on a default install). Auto-compact reads it from Claude Code itself; it runs only in terminal Claude Code, does nothing while a background task runs or within 6 turns of the last cut, and its busy check needs the `background_tasks` list in Claude Code's Stop payload (confirmed in 2.1.300): without it nothing is cut, and gt says so. `/gt:gt-minimize compact` needs Claude Code 2.1.296 or newer; below that nothing cuts and Claude tells you to type `/compact`. With `cut_kind clear` the in-session mod runs `/clear` through Claude Code's command API, at the same size `auto_compact` cuts at (the `auto_clear` policy keeps a shadow record, and clears only once `on` is allowed and was confirmed with the typed word; never while a background task runs or once your next turn has started); that has not yet been seen in a live Claude Code window.
- Long output is kept out by a hook that runs after a successful shell command; the files are private to you and pruned after 7 days. `gt_spill.py run -- <command>` does the same for a command that fails.
- Held-back pastes are scanned first, then saved to private files under `~/.claude/golden-thread/intake/` with any line that looks like a credential replaced by a `[gt: line N withheld ...]` marker (the block message says how many; sending the message again sends them too). Long command outputs are masked the same way. If the scan cannot run, nothing is saved and your message or output goes through as it is. Claude Code still keeps the prompt in your own prompt history.
- `/gt:gt-settings explain <name>` prints the full technical description of any setting.